

02

Matrices

Matrices

Not Just a Table

2.1 Matrix



Core skills you will master in this section:

- ▶ Matrix structure: a two-dimensional array made up of rows and columns, used for data storage and linear transformations.
- ▶ Row and column indexing rules: rows are numbered top to bottom, columns are numbered left to right, and NumPy indexing starts at 0.
- ▶ Matrix indexing and partitioning: get comfortable using NumPy to retrieve row vectors, column vectors, and individual elements.
- ▶ Main diagonal: the main diagonal elements sit where the row index equals the column index.
- ▶ Matrix addition and subtraction: two matrices of the same shape can be added or subtracted element-wise, satisfying the commutative and associative properties.
- ▶ Scalar multiplication: multiplying a matrix by a scalar scales every element up or down.
- ▶ Tensors: a scalar is a 0th-order tensor, a vector is 1st-order, a matrix is 2nd-order, and color images and video are higher-order tensors.

A matrix is a table of numbers arranged in rows and columns, and it's widely used in mathematics, data analysis, machine learning, artificial intelligence, and more. In this section, we define a matrix's row and column structure, and use examples to show how a matrix can represent data. We then discuss matrix addition and subtraction, that is, element-wise addition or subtraction of matrices with the same shape. Next, we introduce scalar multiplication of a matrix, that is, multiplying every element of a matrix by a number. Finally, this section ties together the concepts of scalar, vector, and matrix from the perspective of tensors.

What Is a Matrix?

A matrix is a two-dimensional array arranged in rows and columns. Put simply, a matrix is a table filled with numbers.

A matrix isn't just a mathematical concept — it shows up widely in all kinds of data in everyday life.

For example, we can use a matrix to represent a student report card

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

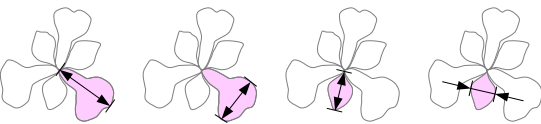
The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

$$\begin{bmatrix} 98 & 90 & 78 \\ 88 & 96 & 80 \\ \vdots & \vdots & \vdots \\ 68 & 89 & 88 \end{bmatrix}$$

Each row of this table represents a student, each column represents a subject, and the values in the matrix are the students' grades.

As another example, the iris dataset in Figure 1 can also be thought of as a data matrix.



Index	Sepal length X_1	Sepal width X_2	Petal length X_3	Petal width X_4	Species C
1	5.1	3.5	1.4	0.2	Setosa C_1
2	4.9	3	1.4	0.2	
3	4.7	3.2	1.3	0.2	
...	...	...	...	...	
49	5.3	3.7	1.5	0.2	
50	5	3.3	1.4	0.2	Versicolor C_2
51	7	3.2	4.7	1.4	
52	6.4	3.2	4.5	1.5	
53	6.9	3.1	4.9	1.5	
...	...	...	...	...	
99	5.1	2.5	3	1.1	Virginica C_3
100	5.7	2.8	4.1	1.3	
101	6.3	3.3	6	2.5	
102	5.8	2.7	5.1	1.9	
103	7.1	3	5.9	2.1	
...	...	...	...	...	
149	6.2	3.4	5.4	2.3	
150	5.9	3	5.1	1.8	



Figure 1. Iris Data, Numeric Values in Centimeters (cm), Image from Coding Made Easy

The dataset's full name is Anderson's Iris data set — iris samples collected by botanist Edgar Anderson on the Gaspé Peninsula in Quebec, Canada.

Each row in the dataset represents one iris sample, and the first four columns represent the iris's four features: sepal length, sepal width, petal length, and petal width.

Figure 1's last column is the iris species. The iris dataset has three different species — Setosa, Versicolor, and Virginica. There are 150 samples in total, 50 for each species.

Figure 2 is a photo of an iris taken by the author. After converting this photo to black-and-white, it can be represented as a 2990×2714 matrix — that is, 2990 rows and 2714 columns — where each element represents a pixel's grayscale value.

Figure 2's photo is clearly not a vector image.

If we keep zooming in, we'll see the edges of the image gradually become blurry. Zooming in further, we can see that the photo is actually a heatmap made up of individual grayscale pixels.

Going further, if we extract 4 pixels from it, corresponding to 4 elements in the matrix, we get a 2×2 real-valued matrix.

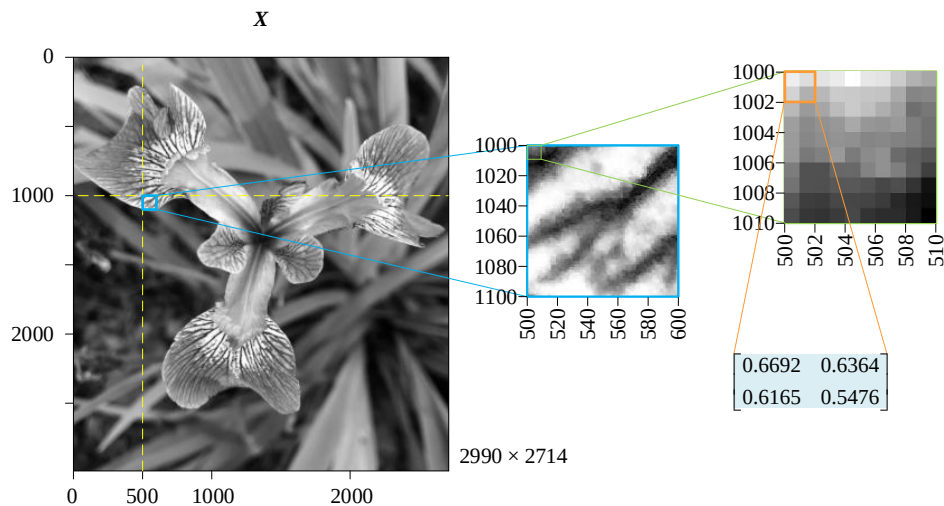


Figure 2. A Photo Is Also a Data Matrix, Image from The Power of Matrices

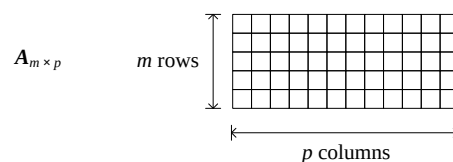
But a matrix is more than just an ordinary table for storing data — you'll see this for yourself very soon.

The Shape of a Matrix

In this book, if a matrix A 's shape is denoted $m \times p$, m represents the number of rows and p represents the number of columns.

This matrix is also often written as $A_{m \times p}$, with the shape in the subscript. The product $m \times p$ is the number of entries (also called components or elements) in the matrix.

As shown in Figure 3, in this book every element of a matrix is visualized as a square cell.

Figure 3. The Shape of Matrix A

As an analogy, the 2×3 matrix A below is like a numerical building 2 stories tall with 3 rooms per floor; while the 3×2 matrix B has 3 stories, with 2 rooms per floor.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

⚠ Note the order — rows first, then columns. This notation hints at the matrix's “direction of growth.”

As mentioned in the previous chapter, row vectors and column vectors are both special shapes of matrices.

A row vector is a matrix with one row and multiple columns, while a column vector is a matrix with multiple rows and one column.

A square matrix is a matrix whose number of rows equals its number of columns.

Rows

The row index is used to locate a row's position in a matrix.

As shown in Figure 4, we use i to denote matrix $\mathbf{A}$'s row index; i increases from top to bottom, ranging over the positive integers from 1 to m .

In this book, matrix $\mathbf{A}$'s row 1 is written as $\mathbf{a}^{(1)}$, row 2 is written as $\mathbf{a}^{(2)}$, and row i is written as $\mathbf{a}^{(i)}$.

⚠ Once again: the row index increases from top to bottom.

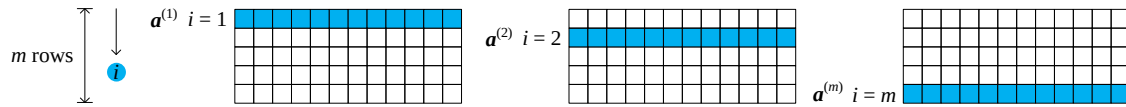


Figure 4. Matrix $\mathbf{A}$'s Row Index

Looking at it from another angle, as shown in Figure 5, a matrix can be viewed as made up of an ordered set of row vectors.

For example, matrix $\mathbf{A}$ can be written as

$$\mathbf{A}_{m \times p} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,p} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,p} \\ a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,p} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,p} \end{bmatrix} \\ \begin{bmatrix} a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,p} \end{bmatrix} \\ \begin{bmatrix} a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,p} \end{bmatrix} \\ \vdots \\ \begin{bmatrix} a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,p} \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \mathbf{a}^{(1)} \\ \mathbf{a}^{(2)} \\ \mathbf{a}^{(3)} \\ \vdots \\ \mathbf{a}^{(m)} \end{bmatrix}$$

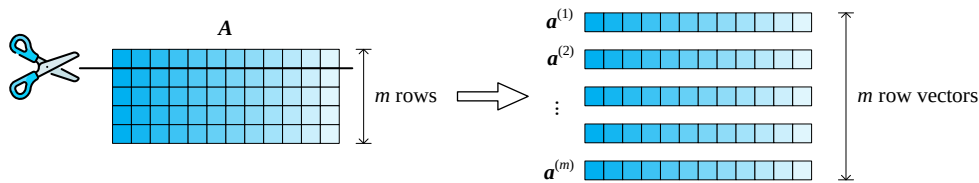


Figure 5. Slicing Matrix $\mathbf{A}$ into a Set of Row Vectors

For example, the 2×3 matrix $\mathbf{A}$ and the 3×2 matrix $\mathbf{B}$ can each be written as a set of row vectors

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \\ \begin{bmatrix} 4 & 5 & 6 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \mathbf{a}^{(1)} \\ \mathbf{a}^{(2)} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 4 \end{bmatrix} \\ \begin{bmatrix} 2 & 5 \end{bmatrix} \\ \begin{bmatrix} 3 & 6 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \mathbf{b}^{(1)} \\ \mathbf{b}^{(2)} \\ \mathbf{b}^{(3)} \end{bmatrix}$$

This actually reflects the idea of a block matrix (also called a partitioned matrix). A block matrix is a large matrix divided by rows and columns into several smaller matrices, each of which is called a block or submatrix. Block matrices will be covered in more depth later in this chapter.

Code 1 creates a 2D array, equivalent to a matrix, then extracts the row vectors shown in Figure 6.

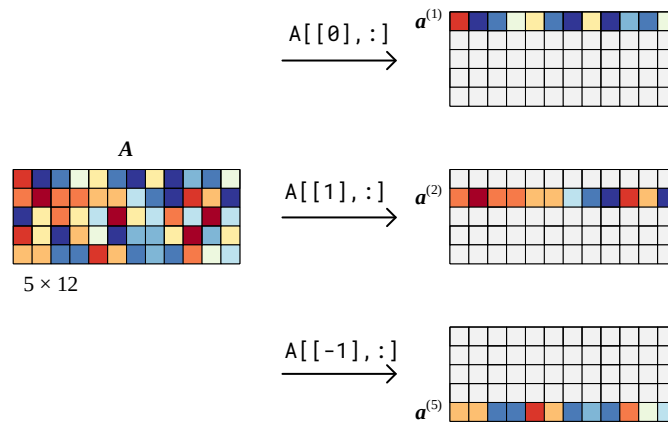


Figure 6. Extracting Row Vectors

Now let's walk through Code 1's key lines.

a `numpy.random.seed(88)` fixes the random number generation, so the same random numbers are produced every time the code runs. 88 is the seed value, which can be any integer; different seeds lead to different sequences of random numbers.

b `numpy.random.randint(0, 10, (5, 12))` generates a 5×12 integer matrix, with values that are random integers between 0 and 9 (note: 10 is excluded). (5, 12) specifies the matrix's shape, where 5 is the number of rows and 12 is the number of columns.

c `seaborn.heatmap()` plots a heatmap to visualize the matrix.

The parameter `cmap='RdYBu_r'` sets the colormap; 'RdYBu_r' goes from red to yellow to blue, reversed (_r).

The parameter `square=True` keeps each cell square.

The parameter `linecolor='w'` sets the gridlines between cells to white.

The parameter `linewidths=0.25` sets the gridline width to 0.25.

d Extracts matrix A's first row and assigns it to the variable `a_row_1`. The index [0] represents the first row, and : represents all columns. Note that `A[[0],:]` returns a 2D array with shape (1, 12), not a 1D array.

e `A[0, :]` likewise gets all the column data for the first row, but returns a 1D array with shape (12,).

f Extracts matrix A's second row and assigns it to `a_row_2`, also returning a 2D array.

g The negative index -1 represents the last row, so this line extracts matrix A's last row and assigns it to `a_row_5`, also returning a 2D array with shape (1, 12).

Code 1. Extracting Rows from a 2D Array | LA_02_01_01.ipynb

```

## 初始化
import numpy as np
import seaborn as sns

## 创建矩阵
a np.random.seed(88)
b A = np.random.randint(0, 10, (5, 12))

## 可视化
c sns.heatmap(A, cmap = 'RdYlBu_r', square = True,
               linecolor = 'w', linewidths = 0.25)

## 行索引
d a_row_1 = A[[0], :]
e A[0, :]

f a_row_2 = A[[1], :]

g a_row_5 = A[[-1], :]

```

Columns

Similarly, the column index is used to locate a column's position in a matrix.

As shown in Figure 7, we use k to denote matrix A 's column index, where k increases from left to right, ranging over the positive integers from 1 to p .

In this book, matrix A 's column 1 is written as $\mathbf{a}_1$, column 2 is written as $\mathbf{a}_2$, and column k is written as $\mathbf{a}_k$.

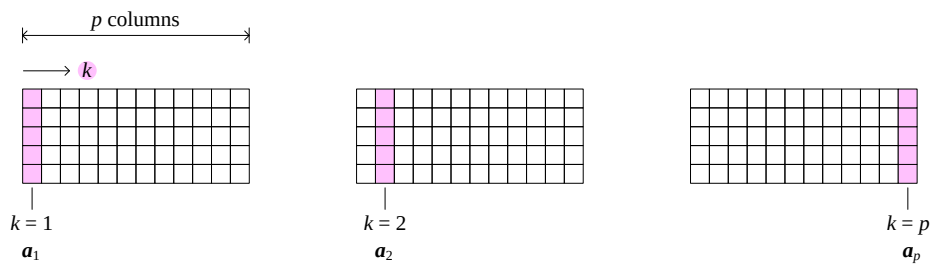
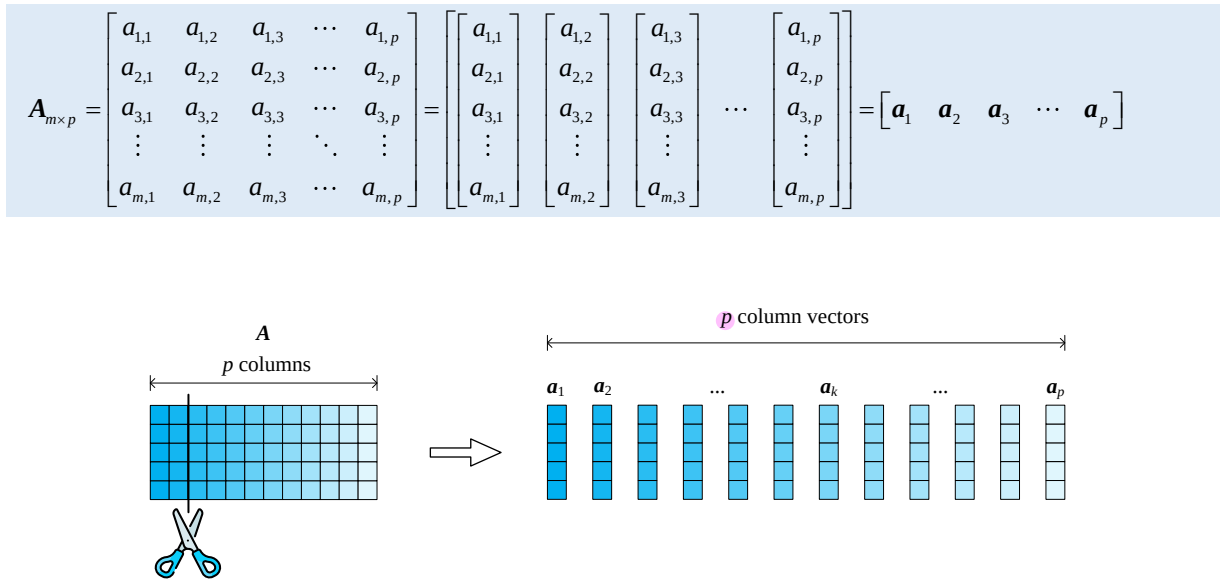


Figure 7. Matrix A 's Column Index

⚠ Once again: in mathematics, a matrix's row and column indices start at 1 and increase from there. But when using a scientific computing library like NumPy, the starting point quietly changes — indexing starts at 0!

Similarly, as shown in Figure 8, a matrix can be viewed as made up of an ordered set of column vectors. For example, matrix A can be written as

Figure 8. Slicing Matrix A into a Set of Column Vectors

For example, the 2×3 matrix A and the 3×2 matrix B can each be written as a set of column vectors

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \left[\begin{bmatrix} 1 \\ 4 \end{bmatrix} \begin{bmatrix} 2 \\ 5 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix} \right], \quad B = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \left[\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \right]$$

This also reflects the idea of matrix partitioning.



LA_02_01_01.ipynb shows how to extract matrix A 's column vectors, specifically as shown in Figure 9.

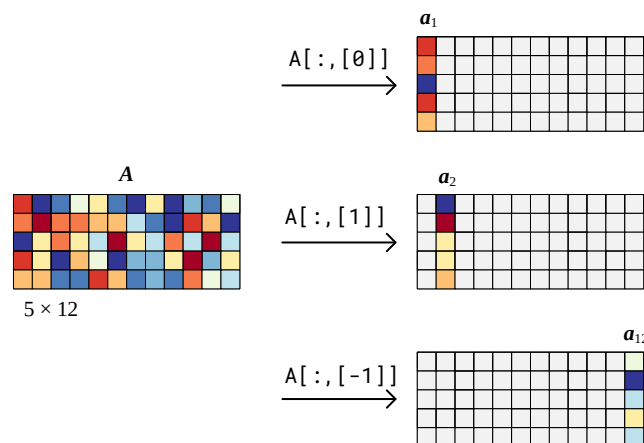


Figure 9. Extracting Column Vectors

Elements of a Matrix

With row and column indices, we can easily label the position of a matrix element.

In general, matrix A 's element in row i , column k is written as $a_{i,k}$ and so, matrix A can be written as

$$A_{m \times p} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,p} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,p} \\ a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,p} \end{bmatrix}$$

For example, matrix A 's element in row 1, column 1 is written as $a_{1,1}$; as another example, matrix A 's element in row 2, column 3 is written as $a_{2,3}$.



LA_02_01_01.ipynb also shows how to get matrix A 's elements, specifically as shown in Figure 10.

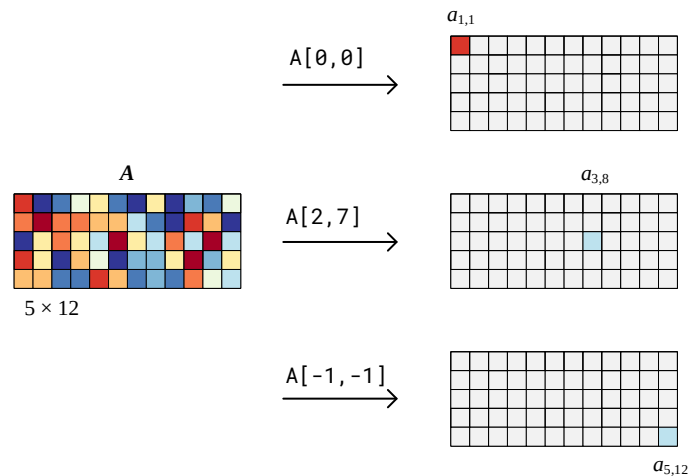
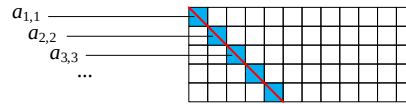


Figure 10. Extracting an Element

Main Diagonal Elements and the Main Diagonal

Every matrix has a special set of elements — the diagonal elements (main diagonal elements) — whose row index equals their column index.

For example, Figure 11's blue elements correspond to matrix A 's diagonal elements $a_{1,1}$, $a_{2,2}$, $a_{3,3}$, and so on.

Figure 11. Matrix A 's Diagonal Elements

The line connecting the diagonal elements is called the main diagonal (also called the diagonal, principal diagonal, or leading diagonal) — that is, the red line in Figure 11.

Geometrically, if we think of each element of a matrix as a square cell of the same size, then the main diagonal is a line segment that starts from the first cell in the matrix's upper-left corner and extends at a 45-degree angle toward the lower right.

The main diagonal passes through every cell whose row index equals its column index, in order — like a neat staircase running through the matrix.

No matter how a matrix's shape changes, the main diagonal always exists. In Figure 12, the red lines are the main diagonals of a tall-narrow matrix, a flat matrix, a square matrix, a row vector, and a column vector.

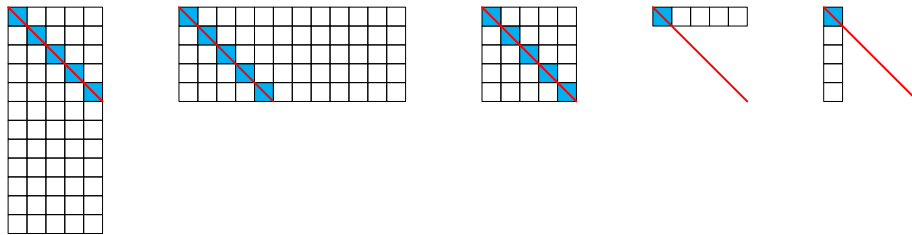


Figure 12. Matrices of Every Shape Have Their Own Main Diagonal

As shown in Figure 13, the function `numpy.diag()` can be used to extract matrix A 's diagonal elements, giving a 1D array as the result.

The function `numpy.diag()` can also be used to create a square diagonal matrix. When passed a 1D array, it generates a square matrix with that array's elements as the diagonal elements, and all other elements filled with zero.

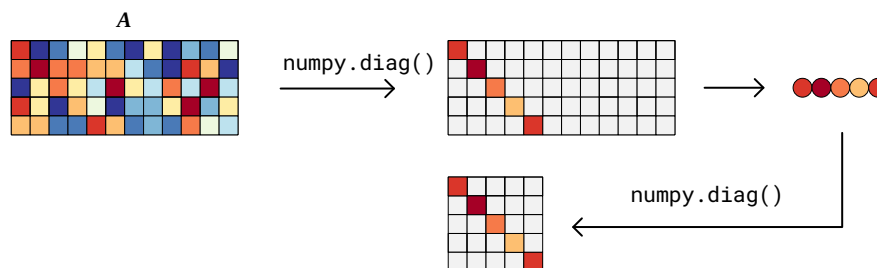


Figure 13. Extracting Diagonal Elements, Giving a 1D Array

Matrix Addition and Subtraction

Adding two matrices $\mathbf{A}$ and $\mathbf{B}$ of the same shape means adding their corresponding elements one by one, as follows:

$$\mathbf{A}_{m \times p} + \mathbf{B}_{m \times p} = \begin{bmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} & \cdots & a_{1,p} + b_{1,p} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} & \cdots & a_{2,p} + b_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} + b_{m,1} & a_{m,2} + b_{m,2} & \cdots & a_{m,p} + b_{m,p} \end{bmatrix}_{m \times p}$$

The commutative property of matrix addition means that for two matrices $\mathbf{A}$ and $\mathbf{B}$ of the same shape, their addition satisfies the following relation:

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$$

That is, no matter which matrix is added first, the result is always the same.

The associative property of matrix addition means that for three matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ of the same size, their addition satisfies the following relation:

$$\mathbf{A} + \mathbf{B} + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) = (\mathbf{A} + \mathbf{B}) + \mathbf{C}$$

That is, summing any two of the matrices first and then adding the third always gives the same result.

The rules above also apply to matrix subtraction, so this book won't repeat them in detail.

In this book, $\mathbf{O}$ denotes a matrix whose elements are all 0, that is, the zero matrix.

The zero matrix has the following property:

$$\begin{aligned} \mathbf{A} + \mathbf{O} &= \mathbf{O} + \mathbf{A} = \mathbf{A} \\ \mathbf{A} - \mathbf{A} &= \mathbf{O} \end{aligned}$$

In the equation above, $\mathbf{A}$ and $\mathbf{O}$ have the same shape. That is, adding any matrix to a zero matrix of the same shape leaves the original matrix unchanged.

Scalar Multiplication of a Matrix

Scalar multiplication of a matrix means multiplying every element of the matrix by a scalar.

For example, the product of scalar k and matrix $\mathbf{A}$ is written as $k\mathbf{A}$:

$$k\mathbf{A} = \begin{bmatrix} k \cdot a_{1,1} & k \cdot a_{1,2} & \cdots & k \cdot a_{1,p} \\ k \cdot a_{2,1} & k \cdot a_{2,2} & \cdots & k \cdot a_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ k \cdot a_{m,1} & k \cdot a_{m,2} & \cdots & k \cdot a_{m,p} \end{bmatrix}$$

Matrix $\mathbf{A}$'s shape is $m \times p$, and the result of scalar multiplication also has shape $m \times p$; every element in the result is scaled up or down.

In particular, when $k = 0$, the result of the equation above is the zero matrix $\mathbf{O}$, with shape $m \times p$.



LA_02_01_02.ipynb shows how to perform matrix addition, subtraction, and scalar multiplication with NumPy; the code is simple, so please study it on your own.

Tensors

A tensor is a multidimensional array in mathematics, describing numerical data across multiple dimensions.

Depending on a tensor's order, it can express information of different levels of complexity:

- ▶ A 0th-order tensor is a scalar.
- ▶ A 1st-order tensor is a vector.
- ▶ A 2nd-order tensor corresponds to a matrix.
- ▶ A tensor of order 3 or higher is a multidimensional array.

Let's return to the earlier photo example. A 0th-order tensor could be the grayscale value of a single pixel in a black-and-white photo, say 0.5.

An example of a 1st-order tensor is an RGB color, which can be represented as the column vector $[R, G, B]^T$.

A D-dimensional space can be expressed as $\mathbb{R}^D$. For example, a 2D plane can be written as $\mathbb{R}^2$, and a 3D space can be written as $\mathbb{R}^3$.

The space of the iris dataset's 4 features is written as $\mathbb{R}^4$, and the space of its 150 samples is $\mathbb{R}^{150}$.

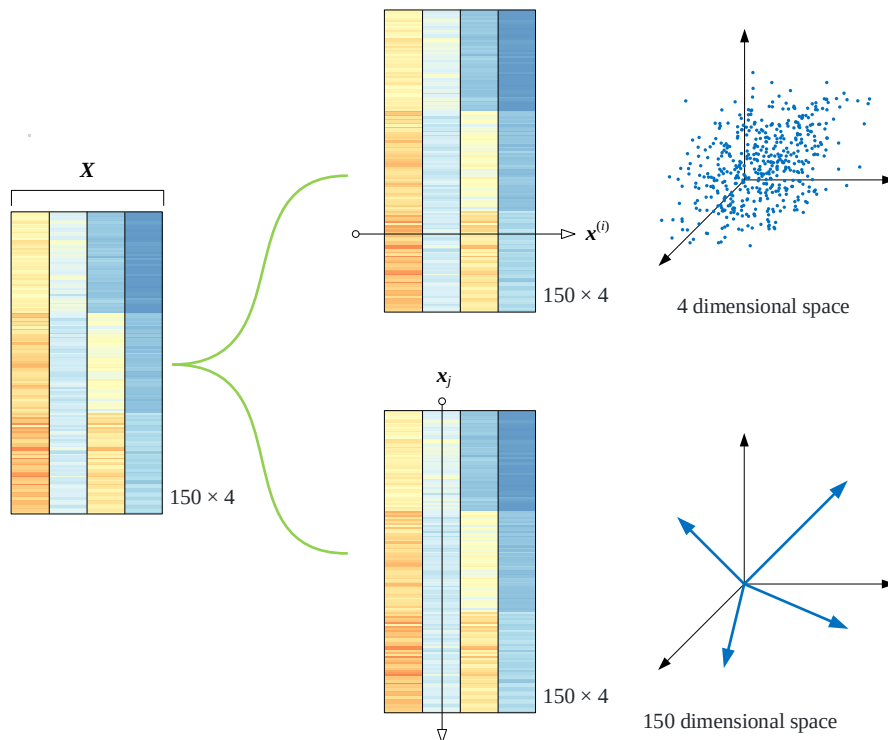


Figure 14. Iris Data from the Row and Column Perspectives, Image from The Power of Matrices

For example, the data of a black-and-white photo is a 2nd-order tensor, with two dimensions given by the photo's height H and width W , written as $\mathbb{R}^{H \times W}$. As another example, a matrix with 2 rows and 3 columns belongs to $\mathbb{R}^{2 \times 3}$.

As shown in Figure 15, a color photo contains information across three channels — R, G, and B — making it a 3rd-order tensor. Compared to a black-and-white photo, a color photo adds a color channel C alongside height H and width W , written as $\mathbb{R}^{C \times H \times W}$. Each color slice of the color photo is then a 2nd-order tensor.

A video, meanwhile, can be viewed as a time series of consecutive color images — a 4th-order tensor that further adds a time (frame) dimension T , written as $\mathbb{R}^{T \times C \times H \times W}$.

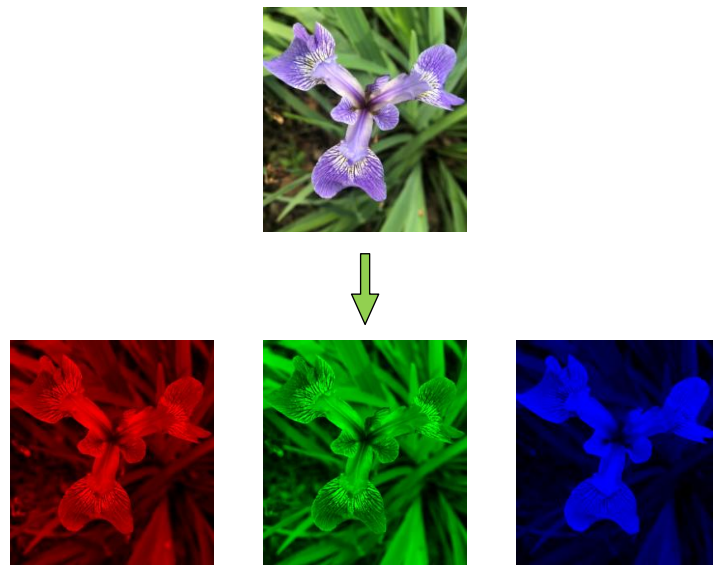


Figure 15. A Color Iris Photo (3rd-Order Tensor) Decomposed into Red, Green, and Blue Channels (Each a 2nd-Order Tensor)

Tensors are a core tool in modern data analysis and machine learning; their ability to represent multidimensional data lays a solid foundation for model building and complex computation. As a natural extension of scalars, vectors, and matrices, tensors can flexibly describe data structures of any dimension, from a single value to multi-channel images, video sequences, and even multimodal data. Many core elements in machine learning — such as a neural network's weight matrices, the pixel-channel representation of images, and word embedding vectors in natural language processing — all exist in the form of tensors.

Through tensors, complex mathematical operations such as matrix multiplication and convolution can be carried out efficiently, and parallelized on hardware such as GPUs, substantially boosting the ability to process large-scale data. Whether in computer vision, natural language processing, or time-series analysis, tensors — with their powerful expressive and computational capabilities — have driven machine learning's broad move from theory into practice, becoming an indispensable foundational tool for building modern artificial intelligence systems.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

Q1. Learn to use `seaborn.heatmap()` to plot a heatmap.

<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Q2. What is a Pandas DataFrame? How is it related to, and different from, a NumPy array?

Q3. How do you construct a Pandas DataFrame from a NumPy 2D array?

Q4. Use a random number generator to create an 8-row, 18-column array, and visualize it with `seaborn.heatmap()`.

Q5. Please extract, from the array in Q4,

- ▶ Row 1 (result should be a 2D array);
- ▶ Row 5 (result should be a 1D array);
- ▶ The last row (result should be a 2D array);
- ▶ Column 1 (result should be a 1D array);
- ▶ Column 5 (result should be a 2D array);
- ▶ The last column (result should be a 2D array);
- ▶ The element in row 1, column 1;
- ▶ The element in row 2, column 2;
- ▶ The element in row 3, column 3;
- ▶ The element in row 8, column 8;
- ▶ All diagonal elements.

Q6. Use a random number generator to create two 3-row, 8-column arrays, and compute their sum and difference.

Q7. Study NumPy's broadcasting mechanism on your own.

<https://numpy.org/doc/stable/user/basics.broadcasting.html>

Q8. Use a random number generator to create an 8-row, 8-column array, and compute the maximum, minimum, and average of each column.

Q9. Please learn to use the following NumPy functions, and explain what each one does.

- ▶ `numpy.flatten()`
- ▶ `numpy.column_stack()`
- ▶ `numpy.row_stack()`
- ▶ `numpy.flip()`
- ▶ `numpy.fliplr()`
- ▶ `numpy.flipud()`
- ▶ `numpy.reshape()`

Q10. Please study the matrix entrywise product (Hadamard product, element-wise product) on your own.

